



CLAUDE CODE & C

SICHER UND PRODUKTIV

ABLAUF & VORBEREITUNG

2 Wochen VORHER

Vorbereitungsgespräch — welche Use Cases & Tagesgeschäft in den Workshop einfließen

1 Woche VORHER

Remote-Setup — Installation gemeinsam
Veranstaltungstag startklar

VERANSTALTUNGSTAG

Der Halbtagesworkshop — Module A–C
(USB-C / HDMI)

1 Woche DANACH

90-Min Praxis-Recap — die interessantesten
gemeinsam gelöst

ZIELE



Was nehmen Sie aus dem Workshop mit?

- Effizientes Vibe Coding (damit wir bekommen, was wir brauchen)
- Ein geregelter Ablauf (etwas Neues bekommen, ohne dass es nervig ist)
- Autonomes Programmieren (ohne dass dabei etwas schief geht)

AGENDA

MODUL A

SICHER LOSLEGEN (ERSTE SCHRITT)

MODUL B

PROJEKTGEDÄCHTNIS (KONFIGURATION)

MODUL C

SKILLS & AUTOMATISCHE LEITPLAN

HANDS-ON-PROGRAMMING MIT SP

A decorative wavy line in a dark blue color, spanning the width of the page.

MODUL A

SICHER LOSLEGEN

CLAUDE CODE **VS.** OPENCODE

CLAUDE CODE

Perfekt für den Einstieg

- Von Anthropic, optimiert auf Claude-Modelle
- Einfacher Einstieg, native Integration in IDEs
- Reifes Ökosystem mit Skills, Hooks, Subagents

OPENCODE

Ideal für...

- O...
- M...
- In...

DATENSCHUTZKONFORM **eins**

Claude Code so konfigurieren, dass keine Daten unkontrolliert

- Anthropic API ersetzen, z.B. Modelle im Green Data Center
- Telemetrie und Daten-Sharing deaktivieren
- Alternativ sensible Dateien und Secrets aus dem Kontext h

```
# Claude Code auf ein lokales Modell umleiten
$ export ANTHROPIC_BASE_URL="http://api.openhippo.io"
$ export ANTHROPIC_AUTH_TOKEN="$HIPPO_TOKEN"
$ export ANTHROPIC_MODEL="hippo-chat-large"
$ export ANTHROPIC_SMALL_FAST_MODEL="hippo-chat-large"
$ export DISABLE_TELEMETRY=1
$ claude
```

Der **nackte** START



Vom leeren Terminal zum ersten echten Code.

- Claude Code oder OpenCode, was passt zu mir?
- Wie konfiguriere ich meinen Helfer (z.B. KI-Modelle auswä
- Was sieht die KI und wie informiert sie sich?

DIE SACHE MIT DEM **Kontext**

Was das Tool weiß, entscheidet über das Ergebnis.

- Was landet im Kontext?
- Effizient bleiben: Ein Chat für eine Aufgabe.
- **Headroom** — das Kontextfenster bewusst nicht überfüllen
- **Codegraph** — bei großen Projekten nur relevante Dateien und Abhängigkeitsgraphen laden

REMINDER Je spezialisierter der Kontext, umso besser das Ergebnis

WIE BAUE ICH MIR eine APP

- 1 Design
- 2 Claude Code / OpenCode Setup
- 3 In kleinen Schritten bauen
- 4 Testen, prüfen, ausliefern

REMINDER Human-in-the-Loop

A decorative wavy line in a dark blue color, spanning the width of the page.

MODUL B

PROJEKTGEDÄCHTNIS

WAS IM **GEDÄCHTNIS** BLEIBEN

An einem Ort — beim nächsten Aufruf sofort verfügbar.

- Projektstruktur
- Architekturentscheidungen
- Sicherheitsregeln
- Coding Standards (z.B. git)
- Richtlinien für CI/CD

TIPP Die KI wie einen Junior Developer bei einem Projekt onk

WO LIEGT DAS **GEDÄCHTNIS**?

Eine Datei — beide Werkzeuge lesen sie.

CLAUDE CODE

- CLAUDE.md im Projektwurzelverzeichnis
- Wird bei jedem Start automatisch geladen
- Kann von der KI aktualisiert werden

OP

- N
- Li
- Fu

MERKE Im Zweifel bei mehreren Entwicklern CLAUDE.md als

Wie **MACHE** ICH ein **PROJEKT**

KI nutzen, um KI zu konfigurieren — nicht von Hand schreiben

- Von der KI generieren lassen (`/init`) statt auf der leeren Seite
- Im Alltag verfeinern: „Merk dir das in CLAUDE.md“ nach gut
- Die KI ihre eigenen Regeln zusammenfassen und kürzen lassen
- Kurz & aktuell halten — mit dem Projekt mitwachsen lassen

MERKE Lass die KI ihr eigenes Gedächtnis schreiben — du redest

A decorative wavy line in a light blue color, spanning the width of the page.

MODUL C

SKILLS & LEITPLANKEN

DIE VIER BAUSTEINE

AGENTS

Delegieren

- Halten den High-Level-Prompt sauber
- Orchestrieren Teilaufgaben von A bis Z

SKILLS

Können

- Bündeln Fachwissen als wiederverwendbare Fähigkeit
- Werden bei Bedarf automatisch geladen

COMPONENTS

Wiederverwenden

- Wiederverwenden
- Konfigurieren

HOOVER

Erzwingen

- Prüfen
- Regulieren

MERKE Lass die Bausteine von der KI erstellen — das kann sie

DIE INDIVIDUALISIERUNG

Eigene Erweiterungen — abgelegt in der Ordnerstruktur.

.CLAUDE/

Claude Code

- commands/ — eigene Befehle
- agents/ — spezialisierte Subagents
- hooks & settings.json — Leitplanken

.AGENTS/

OpenAI Agents

- Agents
- agents
- config

Der Dev-Cycle

Ein kleiner Kreislauf, den man immer wieder durchläuft.



...und wieder von vorn

FRAGE Wie organisiere ich die Arbeit?

IM SANDKASTEN **VOLLGAS**

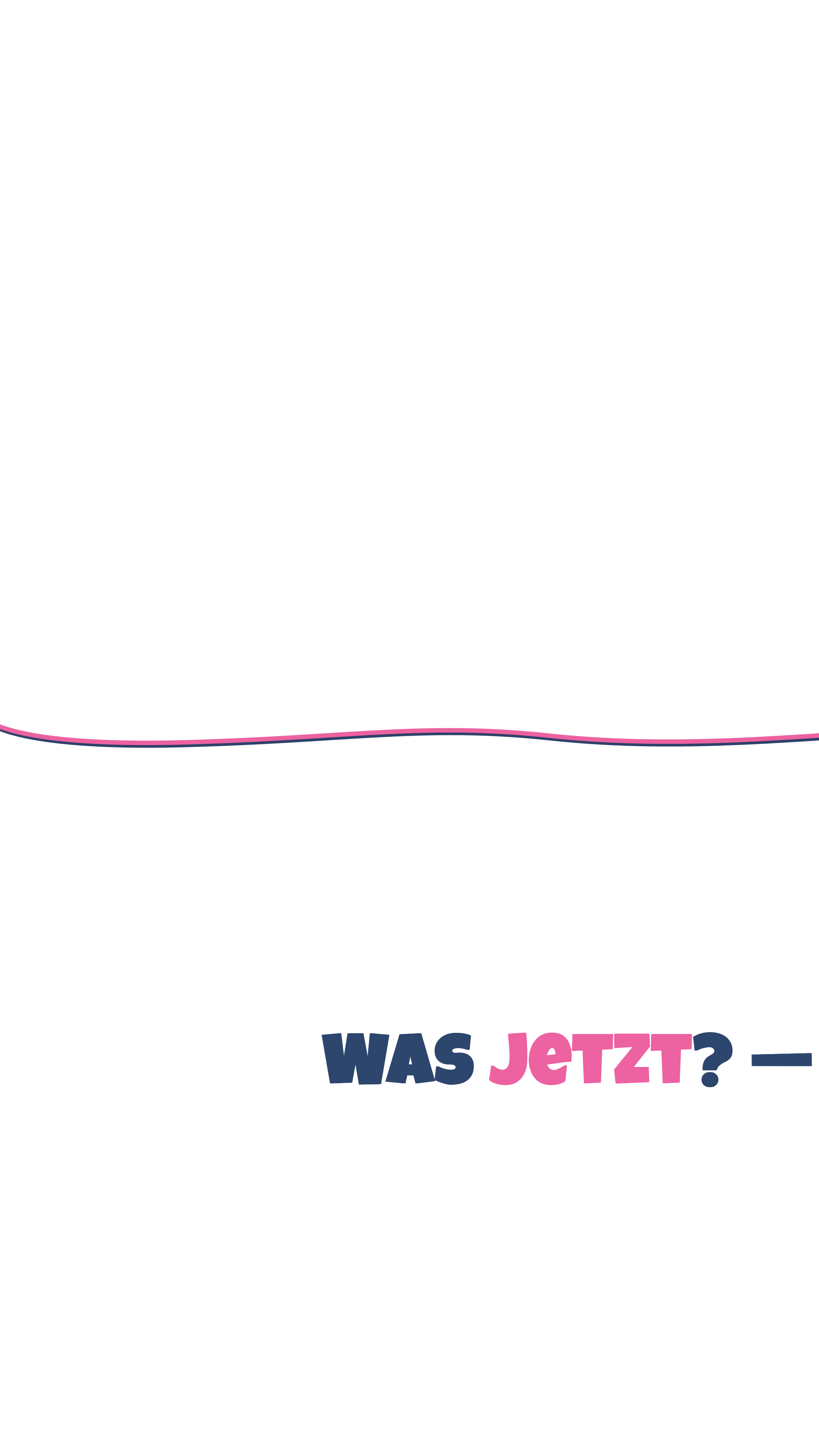
In einer isolierten Umgebung darf die KI ohne Rückfragen a

- Docker-Container: getrennt vom Host, kein Zugriff aufs echte
- Nur das Projekt gemountet — Schaden bleibt eingesperrt un

```
# NUR im isolierten Container ausführen
$ docker run -it --rm -v $PWD:/app sandbox

# Claude ohne Rückfragen
$ claude --dangerously-skip-permissions

# OpenCode: Rechte in opencode.json auf "allow"
$ opencode
```



WAS JETZT? —